

Documentation de `graph_pipeline_prototype.ipynb`

1 Hypothèses métier

Id	Étape	Hypothèse métier	Si elle est fausse
H1	<code>get_customer</code>	Le <code>shortName</code> désigne un client unique, qui possède un <code>publisher</code> dans le namespace <code>artemis-prod</code> .	La jointure <code>inner</code> rend le client invisible, ou <code>collect()</code> renvoie plusieurs <code>db_name</code> et le graphe mélange deux catalogues.
H2	<code>get_products</code>	Le catalogue <i>courant</i> vaut pour toute la fenêtre historique : un produit actif aujourd'hui l'était déjà lors des vues de janvier.	Les produits désactivés depuis sont retirés du graphe alors que leurs vues existaient : le graphe rétrécit à mesure que l'on relance tard.
H3	<code>get_products</code>	Seuls les produits maîtres (<code>parentId = -1</code>) portent le signal ; le tracking a déjà ramené les vues de déclinaisons au maître.	Le signal des déclinaisons est perdu, ou au contraire agrégé alors que les déclinaisons se comportent différemment.
H4	<code>get_products</code>	<code>isActive</code> $\wedge$ <code>isOkInStream</code> = « produit recommandable » ; les autres n'ont pas à être des nœuds.	Des produits recommandés en production sont absents du graphe, donc non scorables.
H5	<code>get_embeddings</code>	Un modèle d'embedding figé (<code>last_model_id</code>) donne au plus une ligne par produit.	Deux embeddings pour le même produit se retrouvent tous les deux dans le résultat, et le remplissage des features en aval choisit l'un des deux de façon non déterministe.
H6	<code>get_views</code>	Un événement <code>product_display_page_event</code> = une vue produit. Pas de pondération par durée ou scroll, pas de déduplication des rafraîchissements.	Un rechargement de page compte comme une vue supplémentaire ; les vues de bots gonflent les sessions.
H7	<code>get_views</code>	<code>t2s_merged_user</code> est fonctionnel (un <code>sourceId</code> $\rightarrow$ un <code>masterId</code>) et l'identité réconciliée est la bonne granularité de session.	Plusieurs <code>masterId</code> pour un <code>sourceId</code> dupliquent les lignes de vues et <i>doublent</i> les poids d'arêtes. Fusionner les appareils enchaîne des navigations parallèles dans une même session.
H8	<code>get_views</code>	Trafic non identifié (<code>user.internalId</code> null) et vue non résolue en produit maître ne portent pas de signal exploitable.	Une part importante du trafic — typiquement les visiteurs non logués — ne contribue à aucune arête.

Id	Étape	Hypothèse métier	Si elle est fausse
H9	<code>get_views</code>	<code>emitted.between(start_date, end_date)</code> : les bornes sont des dates converties en timestamps à minuit, donc la journée du <code>end_date</code> est de fait exclue.	La fenêtre réelle a un jour de moins que prévu ; un décalage d'un jour entre le graphe et un autre calcul qui utiliserait la même variable.
H10	<code>sessionize_views</code>	Une session = 30 minutes d'inactivité (<code>session_timeout</code>), par utilisateur réconcilié, tous appareils confondus. Le tri (<code>emitted, internalId</code>) rend la coupure déterministe en cas d' <i>ex æquo</i> .	Un temps de réflexion long scinde une intention d'achat en deux sessions ; deux intentions rapprochées fusionnent.
H11	<code>split_sessions</code>	Le découpage train/val/test se fait à la granularité de la session (<code>session_start</code>), pas de l'événement individuel.	Une session commencée avant <code>cutoff_train</code> et poursuivie après reste entièrement en train : de l'information postérieure à la coupure entre dans l'entraînement.
H12	<code>build_co_view_edges</code>	Deux produits vus dans la même session sont liés, indépendamment de l'ordre, de l'écart de temps et de la distance dans la session.	Une session de navigation exploratoire produit un graphe complet de produits sans rapport ; le signal séquentiel (A puis B) est perdu.
H13	<code>build_co_view_edges</code>	Une session compte une fois par paire (<code>distinct</code> avant la self-jointure) : le poids d'une arête = nombre de sessions distinctes contenant la paire.	Un utilisateur qui compare intensément deux produits ne pèse pas plus qu'une co-vue fortuite au sein d'une session.
H14	<code>build_co_view_edges</code>	<code>USE_SESSION_WEIGHT_NORM = False</code> : toutes les sessions pèsent pareil. Une session de n produits produit $n(n - 1)/2$ paires de poids 1.	Une session de 50 produits (bot, crawler, comparateur) apporte 1225 arêtes contre 1 pour une session de 2 produits : le graphe est dominé par les sessions longues.
H15	<code>build_co_view_edges</code>	Aucun seuil minimal de poids : une co-occurrence unique est une arête à part entière.	Le graphe est majoritairement constitué d'arêtes de poids 1, c'est-à-dire de coïncidences — et ce sont exactement celles qui varient le plus d'un run à l'autre.
H16	<code>build_co_view_edges</code>	<code>L.internalId < R.internalId</code> élimine les boucles et canonise chaque paire dans un seul sens.	Aucune asymétrie de type « B est un accessoire de A » n'est représentable ici ; la duplication dans les deux sens n'a lieu qu'en aval.
H17	<code>remove_train_edges</code>	Une paire évaluée doit être inédite vis-à-vis du graphe d'entraînement.	Sans cela on mesurerait la mémorisation plutôt que la généralisation. Val et test ne sont pas dédoublonnés entre eux : une même paire peut être dans les deux.

Id	Étape	Hypothèse métier	Si elle est fausse
H18	build	<code>df_train_edges</code> is None sert de marqueur implicite « ceci est le split train » ; c'est le seul signal qui déclenche ou non le retrait des paires déjà en train.	Un appel val/test sans passer <code>df_train_edges</code> ne filtre silencieusement rien : aucune vérification n'assure que le split réellement traité correspond à l'usage prévu du paramètre.
H19	build_node_mapping	Le jeu de nœuds est l'union des arêtes de train, val et test passées en argument.	Des nœuds existent dans le graphe sans aucune arête d'entraînement (isolés), uniquement parce qu'ils ont été co-vus pendant val ou test. Ils augmentent le nombre de nœuds et donc la taille des couches d'embedding.
H20	build_node_mapping	Les index de nœuds sont attribués dans l'ordre croissant des <code>internalId</code> .	Reproductible à jeu de nœuds identique ; mais l'ajout d'un seul produit décale tous les index suivants et rend un fichier de données incompatible avec un <code>node2idx.json</code> d'un autre run.
H21	remap_edges	<code>node2idx</code> couvre déjà tous les <code>internalId</code> de <code>df</code> , car construit en amont sur ce même DataFrame (entre autres).	<code>.map()</code> renvoie NaN pour un <code>internalId</code> absent, silencieusement — la ligne n'est pas retirée mais devient corrompue, et l'erreur ne surviendrait qu'en aval, à la conversion en tensor.
H22	edges_to_tensors	Le graphe de co-vue est traité comme non dirigé : <code>symmetric=True</code> ajoute systématiquement l'arête inverse, complétant la canonisation <code>A < B</code> faite dans <code>build_co_view_edges</code> .	Un appel avec <code>symmetric=False</code> produirait un graphe dirigé avec deux fois moins d'arêtes, sans aucun avertissement puisque le paramètre n'est validé contre rien.
H23	recommended_products_for_t2s_user	Un log ad et une vue produit du même cookie, du même produit et du même client, séparés de moins de 3 secondes, sont le même événement.	Fenêtre trop large : plusieurs vues appartiennent un même log et dupliquent les lignes. Trop étroite : un log est perdu par latence réseau ou décalage d'horloge.
H24	recommended_products_for_t2s_user	Seules les pages <code>PRODUCT</code> portent une recommandation exploitable.	Les recommandations de la page d'accueil, de la recherche et des pages catégorie sont hors périmètre.
H25	positive_negative_edge_construction	L'ordre du tableau <code>relevantProducts</code> est l'ordre d'affichage réel, et seuls les 12 premiers comptent (<code>F.slice(...,1,12)</code>).	Le code prévoit explicitement une variante par score (<code>relevanceScore</code>) ailleurs dans le notebook ; trier autrement changerait quels candidats entrent dans les 12 retenus.

Id	Étape	Hypothèse métier	Si elle est fausse
H26	positive_ negative_ edge_ construction	Positif = candidat recommandé <i>et</i> vu dans la même session — sans contrainte temporelle : un produit vu avant la recommandation compte comme positif.	Fuite temporelle dans la définition du label : une partie des « clics prédits » sont des produits déjà visités, que le modèle peut retrouver par le graphe plutôt que par une vraie prédiction.
H27	positive_ negative_ edge_ construction	Négatif = candidat recommandé, non positif, <i>et</i> absent des arêtes d'entraînement testées dans les deux sens.	Évite d'apprendre contre des faux négatifs, mais retire les négatifs les plus difficiles — ceux qui ressemblent le plus à des positifs.
H28	positive_ negative_ edge_ construction	Asymétrie assumée : les négatifs sont filtrés sur les arêtes du graphe d'entraînement, les positifs ne le sont pas.	Un modèle qui apprend seulement « cette paire est-elle déjà une arête du graphe ? » obtient un bon score sans généraliser.
H29	positive_ negative_ edge_ construction	Négatifs tronqués au rang du positif le plus profond de l'exécution ; les exécutions sans aucun positif sont écartées des arêtes d'entraînement (<code>sample(fraction=0.0, seed=42)</code>).	Le modèle ne voit jamais de liste purement négative, alors que la production en produit. La fraction à 0 est un levier laissé tel quel, pas une propriété du problème.
H30	positive_ negative_ edge_ construction	<code>exec2code</code> couvre toutes les exécutions du split, indépendamment du label, et est rendu stable par tri (<code>all_exec_ids.sort()</code>).	Stable à jeu d'exécutions identique seulement : la moindre exécution en plus ou en moins renumérote tout et invalide les tenseurs déjà écrits avec l'ancien <code>exec2code</code> .
H31	prod_ products_ by_trigger	Aujourd'hui, chaque trigger n'a qu'un seul placement sponsorisé ; le rank final coïncide donc exactement avec la position dans le tableau (1-based).	Si un trigger avait plusieurs placements, la fusion par <code>F.min(pos_rank)</code> et le tie-break par <code>internal_id</code> entreraient en jeu silencieusement, sans qu'aucun test ne le signale aujourd'hui.
H32	add_pos_neg	Pour la comparaison à la production, positif et négatif sont définis par simple appartenance aux produits de la session, sans aucun filtrage sur les arêtes du graphe d'entraînement.	Choix assumé : on veut le vrai retour production, pas des négatifs d'entraînement. Conséquence : la baseline production et le modèle ne sont pas évalués sur exactement la même définition de négatif que celle utilisée à l'entraînement (H26).

Tableau 2 — Hypothèses métier.

2 Ce qui peut être exclu du graphe

Id	Étape	Mécanisme	Ce qui disparaît
E1	get_customer	jointure inner clients / publishers.	Un client sans <i>publisher</i> artemis-prod : le notebook s'exécute alors sur des listes vides sans erreur.
E2	get_products	isActive, isOkInStream	Produits désactivés ou hors flux au moment du run (pas au moment de la vue).
E3	get_products	parentId = -1	Toutes les déclinaisons.
E4	get_embeddings	last_model_id figé	Les embeddings calculés par une autre version du modèle.
E5	get_views	user.internalId non nul	Trafic non identifié.
E6	get_views	masterProductInternalId non nul	Vues non résolues en produit maître.
E7	get_views	emitted.between(...)	Journée du end_date (bornes prises à minuit, voir H9).
E8	build_co_view_edges	self-jointure L.internalId < R.internalId	Sessions à un seul produit distinct : elles ne produisent aucune arête, quel que soit leur volume.
E9	remove_train_edges	left_anti	Paires val/test déjà vues en train.
E10	build_node_mapping	node2idx n'est construit qu'à partir des colonnes product_A/product_B des arêtes fournies.	Les produits catalogués et embeddés qui n'ont jamais été co-vus avec un autre produit ne deviennent jamais des nœuds du graphe, même s'ils ont un embedding valide.
E11	recommended_products_for_t2s_user	pageType = "PRODUCT", fenêtre $\pm 3s$, publisherId isin(...)	Recommandations hors page produit ; logs non appariés à une vue ; logs d'un publisher non résolu.
E12	positive_negative_edge_construction	F.explode(...) sur le tableau des placements sponsorisés	Exécutions sans placement sponsorisé (tableau vide ou nul) : explode les retire silencieusement, contrairement à explode_outer .
E13	positive_negative_edge_construction	F.slice(relevantProducts, 1, 12)	Candidats au-delà du rang 12 — y compris d'éventuels positifs qui s'y trouveraient.
E14	positive_negative_edge_construction	~is_existing_edge	Paires candidates déjà arêtes du graphe d'entraînement, côté négatifs uniquement (H28).
E15	positive_negative_edge_construction	rank_candidate <= deepest_pos_rank	Négatifs plus profonds que le positif le plus profond de l'exécution.
E16	positive_negative_edge_construction	sample(fraction=0.0, seed=42)	Tous les négatifs des exécutions sans aucun positif.
E17	positive_negative_edge_construction	.map(node2idx) + dropna	Paires positives ou négatives dont un des deux produits n'est pas un nœud du graphe (voir H21).

Id	Étape	Mécanisme	Ce qui disparaît
E18	prod_products_by_trigger	filter(rank <= top_k), redondant avec le slice(...,1,12) déjà appliqué en amont tant qu'il n'y a qu'un seul placement par trigger.	Rien aujourd'hui — garde-fou sans effet en placement unique; deviendrait actif si un trigger cu- mulait plusieurs placements.

Tableau 4 — Ce qui peut être exclu du graphe.